

Chapter: Advanced SQL Queries and Techniques

Task Submissions by Day

Table: Tasks

Column	Type
task_id	int
assignee_id	int
submit_date	date

Return:

- `weekend_cnt` → tasks submitted on Saturday or Sunday
- `working_cnt` → tasks submitted on Monday–Friday

Approach 1 — CASE (Best)

```
SELECT
    SUM(
        CASE
            WHEN DAYOFWEEK(submit_date) IN (1,7) THEN 1
            ELSE 0
        END
    ) AS weekend_cnt,

    SUM(
        CASE
            WHEN DAYOFWEEK(submit_date) NOT IN (1,7) THEN 1
            ELSE 0
        END
    ) AS working_cnt
FROM Tasks;
```

Why DAYOFWEEK()?

MySQL returns:

Day	Value
Sunday	1
Monday	2
Tuesday	3
Wednesday	4
Thursday	5
Friday	6
Saturday	7

So

```
DAYOFWEEK(submit_date) IN (1,7)
```

means

Sunday OR Saturday

Consecutive Year Orders

Table: Orders

Column	Type
order_id	int
product_id	int
quantity	int
purchase_date	date

Return the product_id of products that satisfy:

- At least 3 orders in one year.
- At least 3 orders in the next consecutive year.

Self Join (Best)

First count the orders per product per year.

```

SELECT DISTINCT o1.product_id
FROM
(
    SELECT
        product_id,
        YEAR(purchase_date) AS year,
        COUNT(*) AS orders
    FROM Orders
    GROUP BY product_id, YEAR(purchase_date)
) o1
JOIN
(
    SELECT
        product_id,
        YEAR(purchase_date) AS year,
        COUNT(*) AS orders
    FROM Orders
    GROUP BY product_id, YEAR(purchase_date)
) o2
ON o1.product_id = o2.product_id
AND o2.year = o1.year + 1
WHERE o1.orders >= 3
AND o2.orders >= 3;

```

Form a Chemical Bond

Tables

Elements

Column	Type
symbol	varchar
type	enum('Metal','Nonmetal')

Compounds

Column	Type
metal	varchar
nonmetal	varchar

Return all possible compounds by pairing every Metal with every Nonmetal.

Approach 1 — CROSS JOIN (Best)

```
SELECT
    e1.symbol AS metal,
    e2.symbol AS nonmetal
FROM Elements e1
CROSS JOIN Elements e2
WHERE e1.type = 'Metal'
AND e2.type = 'Nonmetal';
```

Explanation

Suppose **Elements**

symbol	type
Na	Metal
K	Metal
Cl	Nonmetal
Br	Nonmetal

Step 1

```
FROM Elements e1 CROSS JOIN Elements e2
```

Every row combines with every other row.

Result

e1	e2
Na	Na
Na	K
Na	Cl
Na	Br
K	Na
K	K
K	Cl
K	Br
Cl	Na
...	...

This is called the Cartesian Product.

If there are:

- 2 metals
- 3 nonmetals

You'll get $2 \times 3 = 6$ possible compounds.

Step 2

Keep only

```
WHERE e1.type='Metal' AND e2.type='Nonmetal'
```

Remaining rows

metal	nonmetal
Na	Cl
Na	Br
K	Cl
K	Br

Exactly the answer.

Approach 2 — JOIN

CROSS JOIN is equivalent to a JOIN with a condition that is always true.

```
SELECT
    e1.symbol AS metal,
    e2.symbol AS nonmetal
FROM Elements e1
JOIN Elements e2
ON 1 = 1
WHERE e1.type = 'Metal'
AND e2.type = 'Nonmetal';
```

Latest Employee Salary

Table: Salary

Column	Type
emp_id	int
firstname	varchar
lastname	varchar
salary	int
department_id	int
salary_date	date

Each employee may have multiple salary records.
Return the latest salary of every employee.

Approach 1 — JOIN + MAX() (Best)

```
SELECT s.*
FROM Salary s
JOIN
(
    SELECT
        emp_id,
        MAX(salary_date) AS latest_date
    FROM Salary
    GROUP BY emp_id
) t
ON s.emp_id = t.emp_id
AND s.salary_date = t.latest_date;
```

Approach 2 — Correlated Subquery

```
SELECT *
FROM Salary s
WHERE salary_date =
(
    SELECT MAX(salary_date)
    FROM Salary
    WHERE emp_id = s.emp_id
);
```

Popularity Percentage

Suppose the table is

user1	user2
1	2
1	3
2	3

Each row means **user1 ↔ user2**

Friendship is bidirectional.

So **1 2** means

- 1 is friend of 2
- 2 is friend of 1

Draw the graph

1
/ 2---3

Now count friends.

- **User 1:** Friends = {2,3}, Count = 2
- **User 2:** Friends = {1,3}, Count = 2
- **User 3:** Friends = {1,2}, Count = 2

Now compute Popularity Percentage

$$\text{Popularity Percentage} = (\text{Number of Friends}) / (\text{Total Users} - 1) \times 100$$

There are 3 users. So denominator is $3 - 1 = 2$

Hence:

- User1: $2 / 2 \times 100 = 100\%$
- User2: 100%
- User3: 100%

Why Total Users – 1 ?

A user cannot be friends with himself.

If there are 5 users, Maximum possible friends = 4.

Hence **Total Users – 1**.

Now let's solve it.

Step 1

```
SELECT user_id,
```

We want one output row per user.

Step 2

```
FROM
(
    SELECT user1 AS user_id
    FROM Friends

    UNION ALL

    SELECT user2
    FROM Friends
) t
```

Why?

Original table

user1	user2
1	2
1	3
2	3

Notice: Every friendship contains two users. We want to count friendships for each user. So we split every friendship into two rows.

First query `SELECT user1` gives

user1
1
1
2

Second query `SELECT user2` gives

user2
2
3
3

Now **UNION ALL** combines them.

Result:

user1
1
1
2
2
3
3

Now every friendship contributes one count to each user.

Why UNION ALL?

Suppose 1 has 10 friends. Then 1 1 1 1 ... must appear 10 times.

If we use **UNION**, duplicates disappear. Then 1 appears only once. Friend count becomes wrong. Hence **UNION ALL**.

Step 3

```
GROUP BY user_id
```

Now MySQL creates one group for every user.

- User1: 1, 1
- User2: 2, 2
- User3: 3, 3

Step 4

```
COUNT(*)
```

Counts rows in every group.

Result:

user	friends
1	2
2	2
3	2

Exactly the number of friends.

Step 5

Now denominator.

```
SELECT COUNT(DISTINCT user_id)
```

Need total number of users. Again Users are stored in two columns. So combine them.

```
SELECT user1 AS user_id FROM Friends  
UNION  
SELECT user2 FROM Friends
```

Notice: This time we use `UNION` NOT `UNION ALL` because we only want unique users.

Result:

user
1
2
3

Now `COUNT(DISTINCT user_id)` returns 3.

Subtract `-1` because a user cannot be friend with himself.

Denominator = 2.

Step 6

Formula: `COUNT(*) * 100 / (total users - 1)`

Example: User1 $2 \times 100 / 2 = 100$

Step 7

`ROUND(..., 2)` Rounds answer to 2 decimal places.

Final Query (Annotated)

```

SELECT
    user_id,                                -- Output one row per
user
    ROUND(
        COUNT(*) * 100 /                    -- Number of friends ×
100
        (
            (
                SELECT COUNT(DISTINCT user_id) -- Count unique users
                FROM
                (
                    SELECT user1 AS user_id
                    FROM Friends

                    UNION                                -- Remove duplicate users

                    SELECT user2
                    FROM Friends

                ) t
            ) - 1                                     -- Exclude the user
himself
        ),
        2                                           -- Round to 2 decimal
places
    ) AS percentage_popularity
FROM
(
    SELECT user1 AS user_id                        -- First endpoint of
friendship

    FROM Friends

    UNION ALL                                       -- Keep every friendship

    SELECT user2                                   -- Second endpoint

    FROM Friends

) t
GROUP BY user_id;

```

Count Occurrences in Text

Table: Files

Column	Type
file_name	varchar
content	text

We need to count how many times the words:

- bull
- bear

appear in all files. The match is case-insensitive.

Approach 1 — REGEXP_REPLACE() (Best)

```
SELECT
  SUM(
    (
      LENGTH(LOWER(content))
      - LENGTH(REGEXP_REPLACE(LOWER(content), 'bull', ''))
    ) / 4
  ) AS bull_count,

  SUM(
    (
      LENGTH(LOWER(content))
      - LENGTH(REGEXP_REPLACE(LOWER(content), 'bear', ''))
    ) / 4
  ) AS bear_count
FROM Files;
```

Explanation

Suppose

`content` : Bull bull Bear

Convert to lowercase: `bull bull bear`

Remove "bull": `bear`

Original length = 14

New length = 6

Difference = 8

Since `bull` has 4 letters

Occurrences = $8 / 4 = 2$

Find Peak Calling Hours for Each City

Table: Calls

Column	Type
caller_id	int
recipient_id	int
call_time	datetime
city	varchar

Return the hour(s) in which each city had the maximum number of calls. If multiple hours have the same maximum count, return all of them.

The simplest non-window solution is a correlated subquery:

```
SELECT
    city,
    HOUR(call_time) AS peak_calling_hour,
    COUNT(*) AS number_of_calls
FROM Calls
GROUP BY city, HOUR(call_time)
HAVING COUNT(*) >= ALL
(
    SELECT COUNT(*)
    FROM Calls c2
    WHERE c2.city = Calls.city
    GROUP BY HOUR(c2.call_time)
);
```